CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

In an increasingly interconnected world, understanding human emotions has become vital for enhancing interactions across various domains, including healthcare, education, entertainment, and customer service. The ability to accurately recognize and interpret emotions can significantly improve user experiences and facilitate more effective communication between humans and machines. However, traditional approaches to emotion recognition often rely on limited modalities, such as text or audio alone, which may fail to capture the full spectrum of emotional expression.

The rise of social media, virtual communication, and immersive technologies has generated a wealth of multimodal data that presents both challenges and opportunities for emotion recognition systems. Text, audio, and visual cues often convey complementary information, and integrating these modalities can lead to a more comprehensive understanding of emotional states. Nevertheless, the inherent complexity of processing and fusing data from different sources complicates the development of effective recognition models.

The integration of knowledge incorporation into feature level fusion is a promising strategy to enhance the performance of multimodal emotion recognition systems. This approach allows the model to leverage domain-specific knowledge, improving the accuracy and relevance of emotion classification. By dynamically weighing the contributions of each modality, these systems can adapt to the unique characteristics of individual interactions, leading to more nuanced and context-aware emotion recognition.

Overall, the advancement of multimodal emotion recognition through the fusion of audio, text, and visual data represents a significant leap forward in understanding human emotions. By harnessing the power of machine learning and knowledge incorporation, these systems offer the potential to transform various applications, creating more empathetic and responsive technologies that enhance human-computer interaction.

1.2 PROBLEM STATEMENT

The reliance on unimodal approaches in emotion recognition systems limits their ability to accurately capture the complexity of human emotions, which are often expressed through a combination of verbal and non-verbal cues. In real-life scenarios, such as healthcare and customer service, understanding emotional states requires analyzing audio, text, and visual signals concurrently.

This lack of comprehensive analysis can lead to misinterpretations and ineffective responses in emotionally charged situations. Therefore, Developing emotion recognition systems that integrate knowledge across multiple modalities is essential to enhance accuracy and contextual understanding, ultimately improving interactions in diverse applications.

1.3 OBJECTIVE

DEVELOP AN AUDIO EMOTION RECOGNITION MODEL: Design and train a deep learning model to identify emotional states using audio data. This model will leverage a hybrid architecture combining CNN, Multi-Head Attention, and GRU, trained on the IEMOCAP dataset to capture emotional nuances in speech.

DESIGN A TEXT EMOTION RECOGNITION MODEL: Create a model for emotion recognition from text data using a CNN architecture that mimics Temporal Convolutional Networks (TCN) for efficient feature extraction. The goal is accurate classification of emotions based on linguistic features.

IMPLEMENT A VIDEO EMOTION RECOGNITION MODEL: Develop a model that analyzes visual data for emotion identification using Spatio-Temporal Graph Convolutional Networks (ST-GCN). This model aims to effectively recognize emotional expressions from visual cues in video sequences.

EVALUATE AND OPTIMIZE EACH MODEL'S PERFORMANCE: Conduct thorough evaluations of each emotion recognition model using performance metrics like accuracy, precision, recall, and F1-score, ensuring effective classification of emotional states in each modality.

CHAPTER 2

LITERATURE REVIEW

The field of multimodal emotion recognition has seen rapid advancements, driven by the need to enhance the accuracy and reliability of emotion detection systems by leveraging diverse data sources such as text, audio, and visual inputs. Recent studies have focused on combining these modalities through advanced fusion techniques and deep learning models, leading to improved performance in recognizing complex emotional states. This literature review explores various approaches, datasets, and technologies used in recent research, highlighting both the progress and challenges faced in building robust multimodal systems.

1. Yijiao Yang et al. [1] proposed a multimodal emotion recognition system based on hybrid attention networks. They employed IEMOCAP, MELD, and RAVDESS datasets for their study. The system extracted features such as MFCCs, chroma features, and spectral contrast for audio, facial landmarks and Action Units (AUs) for visual inputs, and word embeddings from GloVe and BERT for text data. The system was implemented using TensorFlow, OpenCV, and NLP transformers. The results highlighted the effectiveness of attention mechanisms and fusion methods, though challenges related to data dependency were noted.

2. Guee Sang Lee et al. [2] focused on multimodal emotion recognition using Transformer-based fusion and emotion-level representation learning. They utilized IEMOCAP and CMU MOSEI datasets. The features extracted included word embeddings, cropped faces from video frames, and Mel spectrogram chunks from audio signals. Their model, implemented in PyTorch using the ALBERT-base model and MTCNN for face detection, performed well in extracting complex features, though the authors noted limitations in the generalization of hand-crafted features across datasets.

3. Diego Penna et al. [3] developed a framework to evaluate fusion methods for multimodal emotion recognition. They used the IEMOCAP dataset and extracted visual features from 48x48 images processed by VGG19, audio features using MFCC processed by CNN, and textual features from DialogXL. The framework employed Python, Adam Optimizer, and state-of-the-art models for each modality, providing high accuracy but requiring significant preprocessing due to high computational complexity.

4. Hui Ma et al. [4] explored a Transformer-based model with self-distillation for multimodal emotion recognition in conversations, using IEMOCAP and MELD datasets. The features extracted included RoBERTa embeddings for text, openSMILE toolkit features for audio, and DenseNet features for visual inputs. The model was implemented using PyTorch with pre-trained models, and it incorporated self-distillation to improve performance, though complexity in integrating multiple modalities was identified as a challenge.

5. Mani Kumar Tellamekala et al. [5] proposed a COLD Fusion approach for uncertainty-aware multimodal emotion recognition, using the IEMOCAP dataset. Their system extracted 512D feature vectors from visual frames using EmoFAN CNN, but details on audio and text features were not explicitly mentioned. The model utilized PyTorch or TensorFlow and the CMU MultimodalSDK. The study highlighted the system's ability to handle both dimensional and categorical emotion recognition, though high computational complexity was noted.

6. Bogdan Mocano et al. [6] investigated multimodal emotion recognition using cross-modal audio-video fusion with attention and deep metric learning. They employed the IEMOCAP, CMU MOSEI, and MSP-IMPROV datasets, extracting word embeddings and linguistic features for text, MFCCs and pitch for audio, and facial expression features for visual inputs. Their system, implemented using Python, PyTorch, RNNs, LSTMs, CNNs, and deep metric learning techniques, showed superior performance with significant accuracy gains, though the integration of spatial, channel, and temporal attention posed complexity issues.

7. Ma et al. [13] introduce a transformer-based model with self-distillation for multimodal emotion recognition in conversations, addressing the complexities of recognizing emotions in real-world dialogue systems. Their innovative approach highlights the need for adaptive models that can learn from contextual data and improve over time.

8. Moreover, Taghavi et al. [12] focus on improving emotion recognition systems by emphasizing the conversion of speech to text, allowing for better integration of visual and textual data. This methodology highlights the role of non-verbal cues in conjunction with verbal expressions in enhancing emotion recognition accuracy

2.1 CONCLUSION OF LITERATURE SURVEY

2.1.1 FACTORS CONSIDERED IN LITERATURE SURVEY

In the literature, key factors include capturing temporal patterns and contextual features in audio to enhance emotional speech recognition. For motion analysis, the focus is on the spatial and temporal dynamics of human movements to detect emotional cues. In text-based recognition, there is an emphasis on efficiently processing sequential data to ensure accurate feature extraction from sentence structures.

2.1.2 FACTORS NOT UTILIZED ACROSS STUDIES

Several studies have omitted key elements when developing emotion recognition models. In audio processing, some have not fully leveraged advanced techniques like attention mechanisms, which are critical for improving performance. For motion analysis, older approaches that rely on handcrafted features or simpler models were found lacking in capturing the complexity of human movements. Text-based systems sometimes failed to utilize more sophisticated architectures capable of processing sequential data efficiently, relying instead on simpler models like basic RNNs or limited word embeddings.

2.1.3 JUSTIFICATION FOR MODEL CHOICES

TEXT ANALYSIS: CNN-based architectures that mimic Temporal Convolutional Networks (TCN) are chosen for text processing because they efficiently handle sequential data. This architecture captures contextual relationships and temporal dependencies in text, enabling accurate interpretation of emotional sentiment while remaining computationally efficient for real-time processing.

AUDIO PROCESSING: The combination of Convolutional Neural Networks (CNNs), Gated Recurrent Units (GRUs), and Multi-Head Attention is ideal for audio analysis. CNNs capture local acoustic features, GRUs model sequential dependencies, and Multi-Head Attention focuses on various aspects of the audio signal, allowing for a nuanced understanding of emotional speech.

MOTION ANALYSIS: Spatio-Temporal Graph Convolutional Networks (ST-GCNs) are used for motion analysis due to their ability to process graph-structured data. By modeling the spatial relationships between body joints and their temporal dynamics, ST-GCNs effectively detect emotional expressions through human movement.

2.1.4 KEY NOVELTIES OF THE PROPOSED MODELS

TEMPORAL STRUCTURE IN TEXT ANALYSIS: By employing CNNs that has enhanced Temporal characteristics similar to TCN, the text model excels at capturing the evolving context of conversations. This differs from many existing systems that treat text as static, enabling better recognition of emotions that change over time and improving sentiment analysis accuracy.

GRAPH BASED MOTION: The unique aspect of using Spatio-Temporal Graph Convolutional Networks (ST-GCNs) for motion analysis lies in their ability to model human movements as graph-structured data. Unlike traditional methods that rely on fixed spatial representations, ST-GCNs dynamically capture the relationships and interactions between body joints over time. This approach enhances the detection of subtle emotional cues in motion, leading to more accurate emotion recognition compared to standard techniques that may overlook complex movement patterns.

HYBRID APPROACH FOR AUDIO SIGNALS: While many existing models focus solely on either temporal or spectral features, this architecture simultaneously captures local acoustic characteristics through CNNs, maintains sequential dependencies with GRUs, and emphasizes critical audio segments using Multi-Head Attention. This multi-faceted analysis results in a deeper understanding of emotional nuances in speech, offering improved accuracy in emotion recognition compared to conventional audio processing methods.

CHAPTER 3

IMPLEMENTATION

3.1 SYSTEM DESIGN

3.1.1 HARDWARE REQUIREMENTS

1. Minimum RAM: 2GB

   The project requires a minimum of 2GB of Random Access Memory (RAM) to ensure smooth execution of tasks involving data processing, image manipulation, and machine learning operations.

2. Minimum Processor: Core i5

   The project is optimized to run efficiently on systems equipped with at least a Core i5 processor. This ensures adequate computational power for tasks such as image processing, deep learning model training, and inference.

3.1.2 SOFTWARE REQUIREMENTS

1. Operating System Compatibility: Linux, Windows

   The project is designed to be compatible with both Linux and Windows operating systems. This flexibility allows users to deploy and run the software on a variety of platforms based on their preferences and requirements.

2. Python Version: 3.7

   The project is developed using Python programming language version 3.7. Utilizing Python 3.7 ensures compatibility with the required libraries and frameworks, such as OpenCV and TensorFlow, while also providing access to the latest language features and optimizations.

3. Frameworks:

   A. OpenCV:

   OpenCV (Open Source Computer Vision Library) is utilized for various image processing tasks within the project. It provides a wide range of functions and algorithms for tasks such as image filtering, feature detection, object recognition, and more.

B. TensorFlow:

TensorFlow, an open-source deep learning framework developed by Google, is employed for building, training, and deploying machine learning models within the project. TensorFlow offers a comprehensive ecosystem for developing various types of neural networks, including convolutional neural networks (CNNs) commonly used in computer vision applications.

C. Keras:

A high-level API for building and training deep learning models, often used with TensorFlow

3.1.3 MODEL ARCHITECTURE

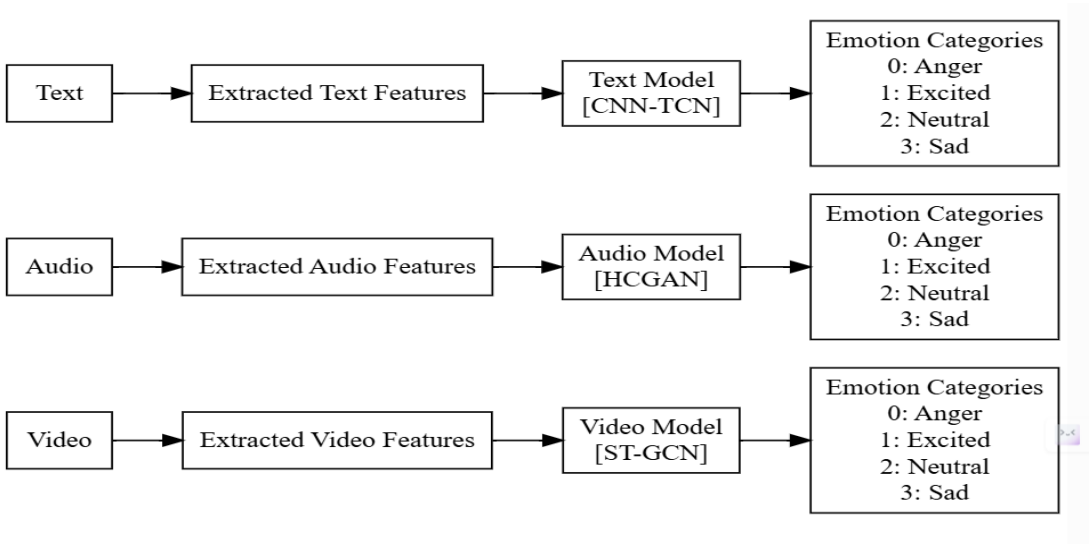

FIG. 1 WORKFLOW OF THE SYSTEM

The proposed emotion recognition model has three distinct modalities: text, audio, and video. Each modality is processed separately, allowing for a comprehensive analysis of emotional cues.

1. Text Processing

a. **Input:** Text data extracted from video.
b. **Model Used: Temporal-Enhanced Convolutional Neural Network (TE-CNN)**
c. **Function:** Analyzes the text to capture emotional context and sentiment.

d. **Output:** Extracted Text Features representing the emotional aspects of the input text.

2. Audio Analysis

a. **Input:** Audio data extracted from video.
b. **Model Used: Hybrid Convolutional Generative Adversarial Network (HCGAN).**
c. **Function:** Identifies emotional tones and nuances in the audio signal.
d. **Output:** Extracted Audio Features that reflect the emotional state conveyed through sound.

3. Video Analysis

a. **Input:** Video frames containing motion data.
b. **Model Used: Spatio-Temporal Graph Convolutional Network (ST-GCN).**
c. **Function:** Interprets spatial and temporal dynamics of human movements and expressions.
d. **Output:** Extracted Video Features capturing visual emotional cues.

4. Emotion Classification Process

a. Each model processes its respective input and generates features independently.
b. The outputs from the models are categorized into one of the following emotion classes:

0: Anger
1: Excited
2: Neutral
3:Sad

3.1.3.1 TEXT MODEL ARCHITECTURE

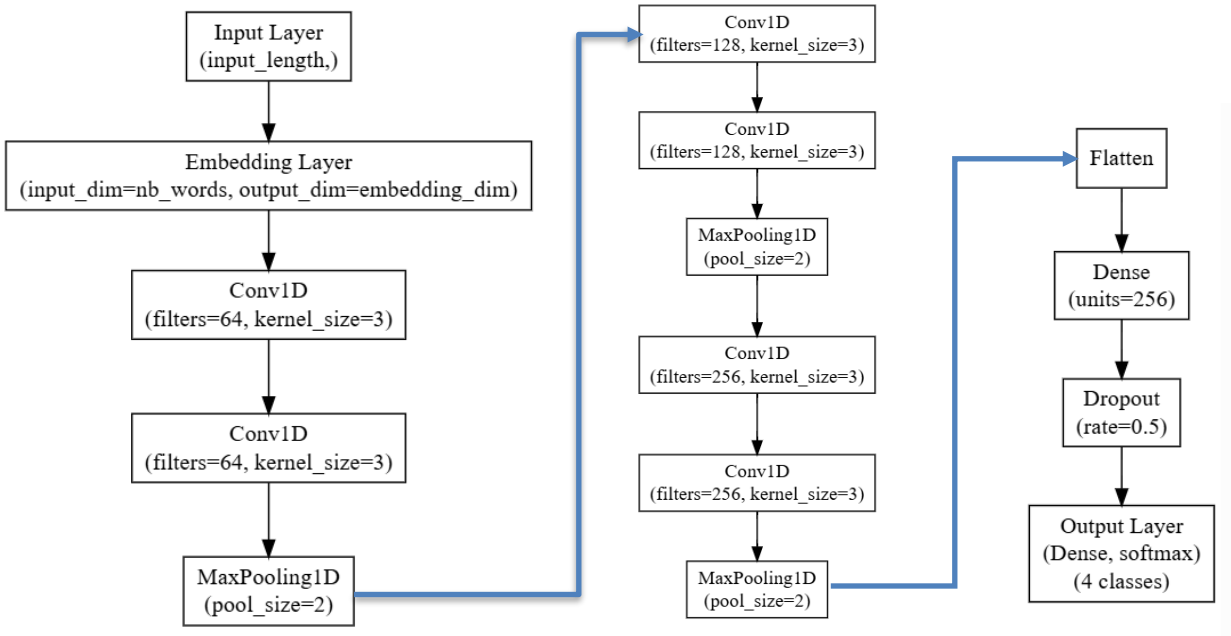

FIG.2 ARCHITECTURE OF TEXT MODEL

This text-based model begins by transforming raw text into numeric indices, which are then converted into dense word embeddings in the embedding layer. These embeddings capture semantic relationships, enabling the model to understand the contextual meanings of words. The Conv1D layers, with progressively increasing filter sizes (64, 128, 256), extract local patterns and important n-grams, making the model particularly adept at recognizing emotional cues in various word combinations. **Mimicking the behavior of Temporal Convolutional Networks (TCN)**, these convolutional layers are key in capturing sequential dependencies in text. MaxPooling1D layers follow each convolution to reduce the dimensionality of feature maps, retaining significant information while reducing computational costs and minimizing overfitting risks. The Flatten layer then prepares the multi-dimensional features for the dense layers by converting them into a 1D vector. The dense layer with 256 units learns complex feature interactions, aiding in effective decision-making. A dropout layer with a 0.5 rate is applied to prevent overfitting by randomly disabling neurons during training, ensuring the model generalizes well to unseen data. Finally, the softmax output layer predicts the probability distribution over the four emotion classes, making it suitable for multiclass classification. This architecture effectively balances feature extraction, temporal pattern recognition, dimensionality reduction, and classification, creating a robust model for emotion detection.

3.1.3.2 AUDIO MODEL ARCHITECTURE

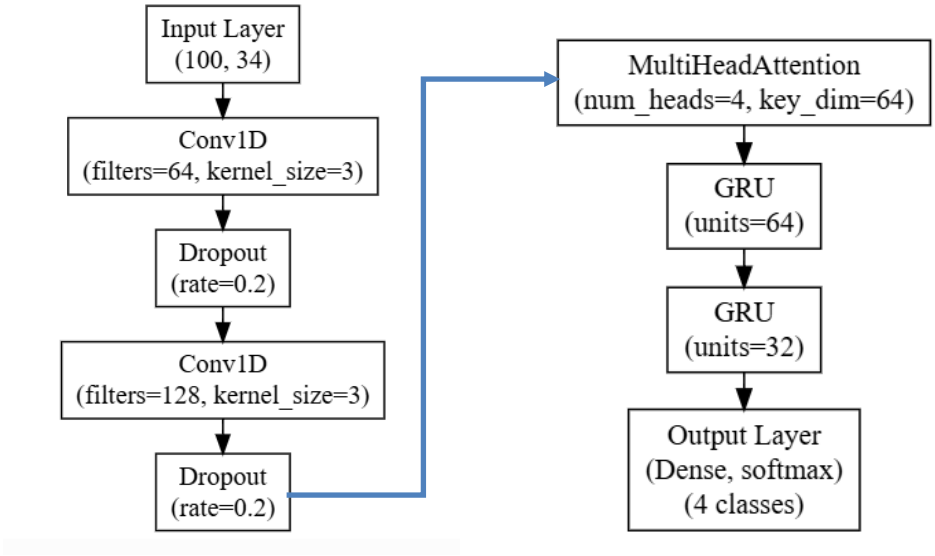

FIG. 3 ARCHITECTURE OF AUDIO MODEL

The **HCGAN** audio-based emotion recognition model begins with an input layer that processes audio features. Two Conv1D layers with increasing filter sizes (64 and 128) apply convolution operations to capture temporal dependencies and key audio patterns relevant to emotional cues. Each Conv1D layer is followed by dropout layers with a 0.2 rate, ensuring regularization by preventing overfitting and improving the model's generalization.

The MultiHeadAttention mechanism (with 4 heads and key dimension size of 64) follows, enabling the model to focus on different parts of the input simultaneously. This helps in capturing complex relationships within the audio sequence. The next stage consists of two GRU (Gated Recurrent Unit) layers with 64 and 32 units, respectively. These GRU layers are responsible for retaining sequential dependencies and extracting high-level temporal features that are crucial in understanding variations in emotional tone over time. Finally, the model outputs the emotion classification through a Dense layer with softmax activation, producing probabilities across four emotion classes. The architecture leverages the strengths of convolutional operations, attention mechanisms, and recurrent layers to ensure an accurate and robust model for audio emotion detection.

3.1.3.2 MOTION MODEL ARCHITECTURE

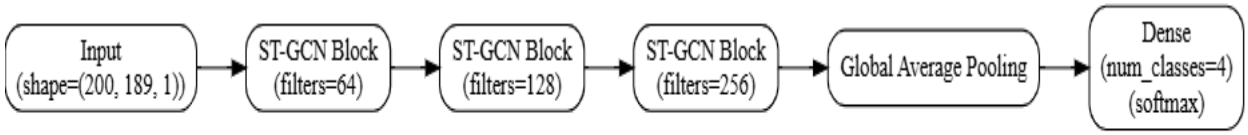

FIG. 4 ARCHITECTURE OF MOTION MODEL

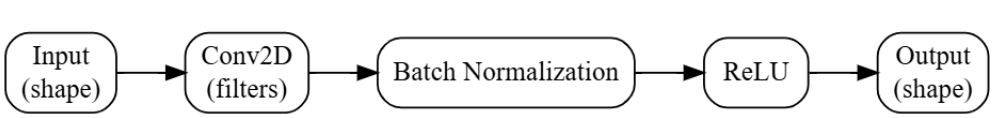

FIG. 5 ARCHITECTURE OF ST-GCN BLOCK

The motion emotion recognition architecture utilizes **Spatio-Temporal Graph Convolutional Networks (ST-GCN)** to effectively analyze motion data, merging concepts from graph theory and convolutional neural networks (CNNs). The process begins with input data, which is structured as a sequence of frames to capture temporal dynamics, followed by multiple ST-GCN blocks that apply graph convolutions to model spatial relationships among points (e.g., joints in skeletal data) while maintaining temporal coherence. These blocks incrementally learn more complex motion patterns through increasing filter sizes, allowing for deeper feature extraction. Global average pooling is then used to condense the output into a fixed-size vector, minimizing overfitting and retaining relevant information. This vector feeds into a dense layer with a softmax activation function that converts the learned features into probabilities for each emotion class, thereby providing a clear categorization based on the motion data. The incorporation of batch normalization and ReLU activations within the blocks further stabilizes training and enhances the model's capacity to represent non-linear relationships, culminating in an architecture well-suited for accurately recognizing emotions expressed through motion.

3.2 SYSTEM IMPLEMENTATION

3.2.1 DATASET DESCRIPTION

1. CONTENT

12 hours of audiovisual data, consisting of 305 videos divided into 5 sessions featuring 10 speakers (2 per session).

2. DOCUMENTATION

Face markers include markers for facial features (e.g., red dots for landmarks) and relationships between them. HumaineInfo.txt identifies the dataset, modalities (audio, video, motion capture), emotions (e.g., happy, sad), and characteristics (10 subjects, English language).

3. PHONETIC RESOURCES

corpus.dic serves as a phonetic lexicon for speech recognition, mapping words to phonetic transcriptions (e.g., ARPAbet). phonemes.txt provides a list of phonemes with examples, useful for speech recognition and synthesis.

4. TIME INFORMATION

timeinfo.txt contains details on time measurements in the dataset, allowing conversion from 10ms to seconds.

5. SESSION DATA

Dialogues and annotations for each session include segments with emotions, VAD (Valence, Arousal, Dominance) scores, and character annotations. Motion capture data files detail hand and head movements captured during dialogues.

a. AUDIO AND VIDEO FILES

The avi folder contains video files for sessions, suitable for multimedia playback. The wav folder includes uncompressed high-quality audio files for each session.

b. TRANSCRIPTIONS

Transcription files hold dialogues for analysis in linguistic research, speech recognition, and conversation analysis.

c. MOTION CAPTURE DATA

This includes structured data representing 3D coordinates for hand movements and other gestures, with each file labeled by session.

This summary highlights the essential components of the IEMOCAP dataset, focusing on its structure, content, and intended use in emotion recognition and related research.

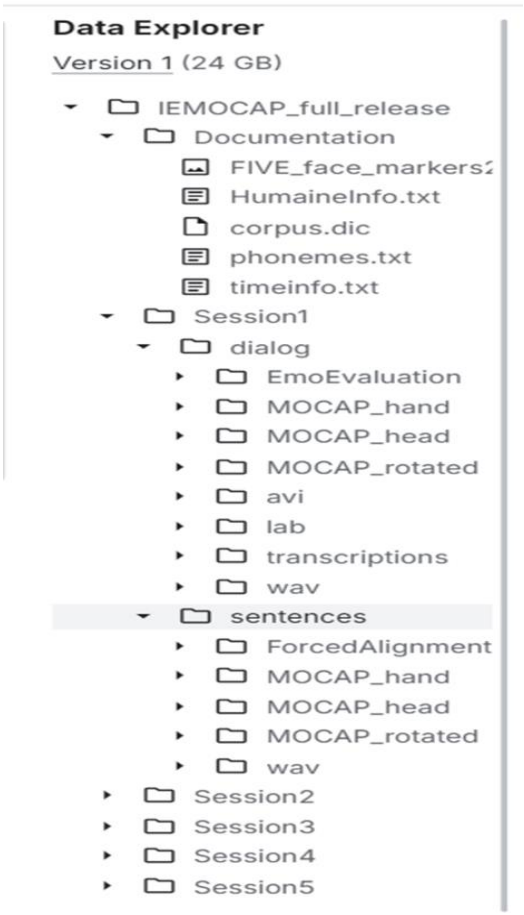

FIG.6 DATASET DETAILS

3.2.2. DATA PREPROCESSING

This section is responsible for extracting, processing, and preparing motion capture (MoCap) and audio data from the IEMOCAP dataset. Below is a breakdown of how each sub-function works in this preprocessing pipeline.

3.2.2.1 HANDLING MOTION CAPTURE DATA

The functions get_mocap_hand, get_mocap_rot, and get_mocap_head are responsible for extracting and processing the motion capture data for hand, rotational movements, and head, respectively. These functions are designed to read the motion capture files, extract the relevant data within a specified time range, and compute the average values.

1. GET_MOCAP_HAND

INPUT: File path to the motion capture hand data, filename, start and end time of the data segment.

PROCESS:

1. Opens the file and skips the header.
2. Iterates through each line, filtering by the time range (start, end).
3. Extracts and stores hand motion data as floating point values.
4. Splits the data into 200 subsets and computes the mean of each subset.
5. Returns the average motion data, or a zero array if no data was found.

2. GET_MOCAP_ROT

INPUT: File path to the rotational motion capture data, filename, start and end time.

PROCESS:

1. Reads the file and skips the header.
2. Filters data lines by the specified time range.
3. Extracts and stores rotational motion data, converts it to float, and computes average values by dividing the data into 200 segments.
4. Returns the processed data or a zero array if no valid data exists.

3. GET_MOCAP_HEAD

INPUT: Path to the head motion capture data, filename, start and end time.

PROCESS:

1. Opens the file and processes each line of data, filtering based on the time range.
2. Converts the extracted data to floating point values and splits it into 200 subsets for averaging.
3. Returns the average head motion data, or a zero array if no data is found.

4. READ_IEMOCAP_MOCAP

INPUT: Path to the IEMOCAP dataset, session identifiers.

PROCESS:

1. Iterates through each session in the dataset.
2. Constructs file paths for audio, emotion annotations, transcriptions, and motion capture data.
3. Reads the .wav audio files, transcriptions, and emotion labels.
4. For each audio file, splits it into segments based on emotion annotations.
5. Extracts motion capture data for hand, rotational, and head movements.
6. Adds the audio, transcription, and motion capture data to the corresponding emotion label.
7. Filters by emotions of interest and checks if an ID has been processed already.
8. Sorts the final consolidated data by ID and returns it as a NumPy array.

This preprocessing pipeline ensures that the raw motion capture and audio data from IEMOCAP is organized and ready for further processing, such as training a multimodal emotion recognition model. This pre-processed data is stored as a pickle file which can be further imported for the upcoming process.

3.2.3 FEATURE EXTRACTION

3.2.3.1 AUDIO MODEL

Feature extraction is a crucial step in audio processing that transforms raw audio signals into a structured format that can be easily analyzed or used for various applications, such as speech recognition, emotion detection, or music analysis. The goal is to capture meaningful characteristics of the audio that represent its content effectively.

Before extracting features, the audio signal undergoes preprocessing:

READING THE AUDIO FILE: The audio file (in WAV format) is read into the system. This step involves loading the raw audio data into memory.

MONO CONVERSION: If the audio is in stereo (two channels), it is often converted to mono (one channel). This simplification is beneficial because many feature extraction methods work better with a single channel, reducing complexity and processing time.

1. SHORT-TERM FEATURE EXTRACTION

Once the audio signal is preprocessed, the next step is to extract features from it. This is typically done in two phases: short-term and mid-term feature extraction.

2. SHORT-TERM FEATURES

Short-term feature extraction involves dividing the audio signal into small, overlapping segments or frames. Each frame is analyzed independently to capture its properties.
Key aspects of short-term feature extraction include:

3. FRAME DURATION: The audio is usually segmented into frames of around 20-50 milliseconds. This allows for the analysis of rapid changes in the audio signal.

4. TYPES OF FEATURES: Common features extracted from each frame include:

a. **Energy:** This indicates the amplitude or power of the audio signal, which can provide insight into loudness.
b. **Zero Crossing Rate (ZCR):** This measures how frequently the signal crosses the zero amplitude line, useful for distinguishing between speech and non-speech segments.
c. **Spectral Features:** These represent frequency characteristics, such as:
    - **Spectral Centroid:** Indicates where the center of mass of the spectrum is located, often correlating with perceived brightness.

- **Spectral Bandwidth:** Measures the width of the spectrum, indicating how spread out the frequencies are.
- **Spectral Roll-off:** This is the frequency below which a certain percentage of the total spectral energy is contained, useful for distinguishing between different types of sounds.

5. MID-TERM FEATURE EXTRACTION

Mid-term feature extraction aggregates the short-term features over longer periods, typically several seconds. This helps to capture more stable characteristics of the audio signal.

1. **Windowing:** The short-term features are analyzed over overlapping windows, typically spanning several seconds. This allows for the assessment of changes over time.
2. **Statistics Computation:** Instead of analyzing every single frame, mid-term features may involve calculating statistical measures (mean, variance, etc.) from the short-term features, providing a more robust representation of the audio's characteristics.

6. BEAT ESTIMATION

In applications like music analysis, estimating the beat or tempo of a musical piece is crucial. The beat estimation process involves:

1. **Analyzing Rhythm Features:** It focuses on features that represent rhythmic aspects of the audio, such as energy fluctuations.
2. **Peak Detection:** By detecting peaks in rhythmic features, the system can identify moments that likely correspond to musical beats.
3. **Beat Histogram:** A histogram of the intervals between detected beats can help estimate the overall tempo of the audio.

7. SAVING EXTRACTED FEATURES

Once the features are extracted, they can be saved for further analysis:

1. **Numpy Files:** Features are often saved in a binary format (like .npy files), which allows for efficient loading and storage of large arrays.
2. **CSV Files:** For ease of use in different applications, features can also be exported as .csv files, which can be easily opened in spreadsheet software or imported into machine learning frameworks.

3.2.3.2 TEXT MODEL

Word embeddings are a representation of text where words with similar meanings have similar representations. These representations are dense vectors of real numbers, as opposed to the sparse vectors used in one-hot encoding. GloVe (Global Vectors for Word Representation) is one of the most widely used pre-trained word embedding techniques.

1. TOKENIZATION AND WORD INDEXING

Before embedding, the text data is typically preprocessed, and one crucial part of that is tokenization. Tokenization splits the text into individual words or tokens. After this, each word is assigned an index, which is stored in the word_index.
The word_index is essentially a dictionary where each unique word in the corpus is mapped to an integer index. This helps in creating a vocabulary for the text data.

2. PRE-TRAINED GLOVE EMBEDDINGS

GloVe embeddings are pre-trained word vectors learned from large corpora of text. These embeddings capture semantic relationships between words. Each word is represented as a vector in a high-dimensional space, typically 50 to 300 dimensions, where the distance between words reflects their semantic similarity.

> **GLOVE EMBEDDING FILE**: The file used here, 'glove.42B.300d.txt', contains pre-trained word vectors with a dimensionality of 300 (hence the name 300d). Each line in this file contains a word followed by its vector representation in the 300-dimensional space.

3. LOADING THE GLOVE EMBEDDINGS

The GloVe embeddings are loaded into a dictionary called gembeddings_index. Each word is stored as a key, and its corresponding vector (a 300-dimensional array of floating-point numbers) is stored as the value.

1. The code opens the GloVe file using UTF-8 encoding.
2. It reads each line, splits the line into the word (first part) and its embedding (rest of the parts).
3. The word is used as a key in the dictionary, and the corresponding embedding is stored as a value.

This dictionary allows quick lookup of the embedding vector for any word.

4. CREATING THE EMBEDDING MATRIX

The next step is to create an embedding matrix, which is a 2D array where each row corresponds to a word from the vocabulary (from word_index), and each column corresponds to one of the 300 embedding dimensions.

> **EMBEDDING MATRIX**: The embedding matrix (g_word_embedding_matrix) has a size of (nb_words, EMBEDDING_DIM) where nb_words is the number of words in the vocabulary plus one (for padding) and EMBEDDING_DIM is 300.
>
> 1. The embedding matrix is initialized with zeros.
> 2. For each word in word_index, the code looks up its embedding in the gembeddings_index. If the embedding is found, the corresponding row in the matrix is filled with the GloVe embedding vector. If the embedding is not found (e.g., for rare or unknown words), the row remains as a zero vector.

This embedding matrix can then be used as input for machine learning models, particularly neural networks, where each word is represented by a 300-dimensional vector.

5. HANDLING OUT-OF-VOCABULARY WORDS

In the embedding matrix, words that do not have pre-trained embeddings in GloVe (out-of-vocabulary words) will have a row of zeros. This helps track how many words in the dataset do not have a corresponding pre-trained embedding.

This process of feature extraction through word embeddings transforms raw text data into a structured numerical format that can be directly used in machine learning models. By leveraging pre-trained embeddings, the model can capture subtle semantic nuances between words, leading to more accurate predictions

3.2.3.3 MOTION MODEL

The motion capture data undergoes preprocessing to extract features from three key components: head motion, hand motion, and body rotation. Each type of motion data is represented as a matrix:

a. Head Motion Features: 18 features are extracted to represent head movements. Each sequence is standardized to have 200 time steps, ensuring consistency across all samples.
b. Hand Motion Features: 6 features capture the movements of both hands. Similar to the head motion data, each sequence is limited to 200 time steps.
c. Body Rotation Features: 165 features describe rotational movements of the body. This data is also standardized to 200 time steps for uniformity.

All missing values are replaced with zeros to handle incomplete data. The final feature set for each sequence is constructed by concatenating the head, hand, and body rotation features, resulting in a combined matrix of shape 200 x 189 for each sample.

3.2.4 FEATURE ENGINEERING

1. LABEL BINARIZATION

Label binarization is a technique used to convert categorical labels into a binary format, which is essential for multi-class classification problems. In this case, the labels represent different emotional states, and the objective is to prepare them for input into a machine learning model.

2. EXTRACTING EMOTION LABELS

The first step involves creating a list of emotion labels from the dataset:

1. **DATA COLLECTION**: The iteration over a dataset, referred to as data2, includes multiple entries, each associated with an emotion label.
2. **EMOTION RETRIEVAL**: For each entry in the dataset, the corresponding emotion is accessed through a specific key. This process aggregates all the emotion labels into a list.

As a result, the list contains all the emotions associated with each data sample, preparing them for the next stage.

3. UNDERSTANDING THE UNIQUE EMOTIONS

Before proceeding with binarization, it's essential to identify the unique emotions that will be used for encoding:

UNIQUE EMOTIONS: A separate variable contains the unique emotion labels that are relevant to the task. This step ensures that the binarization only considers specific emotions intended for classification.

4. BINARIZATION PROCESS

The core of the process is the binarization of the emotion labels:

LABEL BINARIZATION: A method from a popular machine learning library is utilized to convert the categorical emotion labels in the previously created list into a binary matrix. Each unique emotion is transformed into a binary vector:

- If a sample belongs to a particular emotion, the corresponding position in the vector is set to 1; otherwise, it is set to 0.

This transformation enables the model to interpret emotions in a binary format, making it suitable for multi-label classification tasks.

3.2.5 MODEL TRAINING AND VALIDATION

3.2.5.1 AUDIO MODEL

1. AUDIO SIGNAL ANALYSIS

a. **WINDOWING:** To begin, a time window of 0.2 seconds is established to segment the audio signal. This windowing technique allows for localized analysis of the audio over short durations. The number of samples corresponding to this window is calculated based on the audio's sampling frequency, ensuring that each segment contains a consistent amount of data for analysis.

b. **SHORT-TIME FEATURE EXTRACTION**: Features are extracted from the audio frames using the predefined window length and an overlap of half the window size in the method defined in the feature extraction stage. This approach enables the continuous analysis of the audio signal, allowing for the capture of temporal dynamics that are crucial for understanding speech patterns.

c. **DERIVATIVE CALCULATION**: After the initial feature extraction, the process evaluates the shape of the resulting feature matrix. If multiple columns are present, the derivatives of the features are computed by considering the values of adjacent

columns. This derivative information helps capture the rate of change in the features, providing additional insights into the audio characteristics. In cases where fewer columns are extracted, a suitable derivative array is initialized to maintain consistency.

d. **COLLECTING FEATURES**: The extracted features for each audio segment are systematically collected into a list. This collection is essential for preparing the data for subsequent analysis or modeling. To ensure that all features have a uniform input size, the extracted features are padded to match a predetermined maximum length.

e. **PROGRESS TRACKING**: Throughout the extraction process, a counter is utilized to keep track of the number of processed audio segments. This counter updates every 100 iterations, allowing for efficient monitoring and management of the feature extraction workflow.

f. **FINAL OUTPUT**: The **x_train_speech** array is constructed by extracting short-term features from audio segments. Each segment's features are computed and then padded or truncated to a maximum length of 100 units. The resulting features are transposed and collected into the **x_train_speech** NumPy array for subsequent analysis.

2. DATASET SPLIT

The **x_train_speech** dataset is divided into training and testing subsets.

1. **xtrain_sp** contains the first 3,838 entries from **x_train_speech** for training purposes.
2. **xtest_sp** consists of the remaining entries in **x_train_speech** for testing.

The labels in **y_train** are also split accordingly.

1. **ytrain_sp** contains the first 3,838 labels from **y_train** for training.
2. **ytest_sp** includes the remaining labels from **y_train** for testing.

3. MODEL COMPILATION

The model is compiled using the **Adam** optimizer with a learning rate of **0.001**. The loss function used is **categorical_crossentropy**, suitable for multi-class classification tasks. **Accuracy** is selected as the evaluation metric to monitor the model's performance during training.

4. MODEL INITIALIZATION

The **create_model** function is called to initialize the model architecture, configured to accept input shapes of **(100, 34)**.

5. MODEL TRAINING

The model is trained using the **fit** method with the following parameters:

1. **x_train_speech** and **y_train** are used as the training data and labels, respectively.
2. **xtest_sp** and **ytest_sp** are provided as validation data to evaluate the model's performance on unseen data during training.
3. Training is conducted over **65 epochs** with a **batch size of 32**.

6. TRAINING HISTORY

The training process produces a history object containing information about the training and validation metrics for each epoch, which can be utilized for further analysis.

FIG. 7 AUDIO MODEL SUMMARY

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 100, 34)	0	-
conv1d_1 (Conv1D)	(None, 98, 64)	6,592	input_layer_1[0]…
dropout_1 (Dropout)	(None, 98, 64)	0	conv1d_1[0][0]
conv1d_2 (Conv1D)	(None, 96, 128)	24,704	dropout_1[0][0]
dropout_2 (Dropout)	(None, 96, 128)	0	conv1d_2[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 96, 128)	131,968	dropout_2[0][0], dropout_2[0][0]
gru (GRU)	(None, 96, 64)	37,248	multi_head_atten…
gru_1 (GRU)	(None, 32)	9,408	gru[0][0]
dense_2 (Dense)	(None, 4)	132	gru_1[0][0]

Total params: 210,052 (820.52 KB)

Trainable params: 210,052 (820.52 KB)

Non-trainable params: 0 (0.00 B)

3.2.5.2 TEXT MODEL

1. TOKENIZATION

- In the initial phase of preparing text data for machine learning, a maximum sequence length is defined, set at 500 tokens. This parameter ensures that each input sequence will not exceed this length; sequences longer than this will be truncated, while shorter ones will be padded with zeros to maintain uniformity.

- Next, a Tokenizer object is instantiated, which is essential for converting textual data into numerical format. This Tokenizer is fitted on the text data, which involves analyzing the dataset to create a vocabulary of unique words. Each word in this vocabulary is then assigned a unique integer index based on its frequency of occurrence in the text.

- Following this, the Tokenizer is employed to convert the text into sequences of integers. This process results in a list of sequences, where each word in the original text is replaced with its corresponding integer index.

- To ensure that the input sequences can be processed uniformly by machine learning models, the sequences are padded using the pad_sequences function. This function adjusts the length of each sequence to match the specified maximum length by adding zeros to shorter sequences. The final output of this preprocessing stage is the variable **x_train_text**, which contains the padded sequences prepared for subsequent model training.

2. DATASET SPLIT

The **x_train_text** dataset is divided into training and testing subsets.

1. **xtrain_tx** contains the first 3,838 entries from **x_train_text** for training purposes.
2. **xtest_tx** consists of the remaining entries in **x_train_text** for testing.

The labels in **y_train** are also split accordingly.

1. **ytrain_tx** contains the first 3,838 labels from **y_train** for training.
2. **ytest_sp** includes the remaining labels from **y_train** for testing.

3. MODEL COMPILATION

The model is compiled using the **Adam** optimizer, set with a learning rate of **0.001**. **Categorical crossentropy** is employed as the loss function, making it suitable for multi-class classification tasks. Accuracy is selected as the evaluation metric to monitor the model's performance throughout the training process.

4. MODEL INITIALIZATION

The create_simple_tcn_model function is invoked to initialize the model architecture, designed to handle input shapes defined by input_length, embedding_dim, nb_words, and embedding_matrix.

5. MODEL TRAINING

The model is trained using the fit method with the following parameters:

1. **x_train_text** and **y_train** serve as the training data and corresponding labels.
2. **xtest_tx** and **ytest_tx** are utilized as validation data to assess the model's performance on unseen examples during training.
3. Training is conducted for **50** epochs with a batch size of **32**.

6. TRAINING HISTORY

The training process yields a history object, which encapsulates information regarding the training and validation metrics for each epoch, facilitating further analysis and visualization of the model's performance over time.

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 500)	0
embedding_2 (Embedding)	(None, 500, 300)	821,100
conv1d (Conv1D)	(None, 500, 64)	57,664
conv1d_1 (Conv1D)	(None, 500, 64)	12,352
max_pooling1d (MaxPooling1D)	(None, 250, 64)	0
conv1d_2 (Conv1D)	(None, 250, 128)	24,704
conv1d_3 (Conv1D)	(None, 250, 128)	49,280
max_pooling1d_1 (MaxPooling1D)	(None, 125, 128)	0
conv1d_4 (Conv1D)	(None, 125, 256)	98,560
conv1d_5 (Conv1D)	(None, 125, 256)	196,864
max_pooling1d_2 (MaxPooling1D)	(None, 62, 256)	0
flatten (Flatten)	(None, 15872)	0
dense (Dense)	(None, 256)	4,063,488
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 4)	1,028

Total params: 5,325,040 (20.31 MB)

Trainable params: 5,325,040 (20.31 MB)

Non-trainable params: 0 (0.00 B)

Simple TCN-like Model Built

FIG. 8 TEXT MODEL SUMMARY

3.2.5.3 MOTION MODEL

1. DATASET SPLIT

The dataset is divided into training and testing sets:

1. **xtrain_sp** contains the first 3,838 entries from x_train_mocap for training purposes.
2. **xtest_sp** consists of the remaining entries for testing.

The labels in **Y** are split accordingly:

1. **ytrain_sp** contains the first 3,838 labels for training.
2. **ytest_sp** includes the remaining labels for testing.

2. MODEL COMPILATION

The motion model is compiled using the Adam optimizer with a learning rate of 0.001. Categorical crossentropy is selected as the loss function, appropriate for multi-class classification tasks. Accuracy is used as the evaluation metric to monitor the model's performance throughout training.

3. MODEL INITIALIZATION

The model is initialized using a custom architecture based on the Spatial-Temporal Graph Convolutional Network (ST-GCN). The input shape is defined as (200, 189, 1), corresponding to the processed motion capture data.

4. MODEL TRAINING

The model is trained using the **fit** method with the following parameters:

1. **xtrain_sp** and **ytrain_sp** serve as the training data and labels.
2. **xtest_sp** and **ytest_sp** are utilized as validation data to monitor the model's performance on unseen data during training.
3. Training is conducted for 50 epochs with a batch size of 32.

5. TRAINING HISTORY

The training process produces a history object that contains detailed information about the training and validation metrics for each epoch. This data can be further analyzed to track the model's progress and optimize performance over time.

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 200, 189, 1)	0
conv2d (Conv2D)	(None, 200, 189, 64)	640
batch_normalization (BatchNormalization)	(None, 200, 189, 64)	256
re_lu (ReLU)	(None, 200, 189, 64)	0
conv2d_1 (Conv2D)	(None, 200, 189, 128)	73,856
batch_normalization_1 (BatchNormalization)	(None, 200, 189, 128)	512
re_lu_1 (ReLU)	(None, 200, 189, 128)	0
conv2d_2 (Conv2D)	(None, 200, 189, 256)	295,168
batch_normalization_2 (BatchNormalization)	(None, 200, 189, 256)	1,024
re_lu_2 (ReLU)	(None, 200, 189, 256)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 256)	0
dense (Dense)	(None, 4)	1,028

Total params: 372,484 (1.42 MB)

Trainable params: 371,588 (1.42 MB)

Non-trainable params: 896 (3.50 KB)

FIG. 9 MOTION MODEL SUMMARY

CHAPTER 4

EXPERIMENTATION

4.1 EXPERIMENT

Model	Epochs	Training Steps	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy
Existing Models						
LSTM	60	2127	0.9255	0.5971	0.9124	0.6579
LSTM with Attention	75	2127	0.7230	0.6977	0.8288	0.6955
BiLSTM with Attention	100	2034	0.3225	0.8732	1.7675	0.7472
Proposed Model						
Capsule Network (failed)	100	124	0.7809	0.7068	1.2321	0.5202
HCGAN	65	155	0.6319	0.7304	0.4962	0.8179

TABLE. 1 EXPERIMENTATION RESULTS OF AUDIO MODELS

This table 1 presents the performance metrics of various audio-based emotion recognition models, highlighting their training and validation accuracies. The results underscore the effectiveness of different architectures in capturing emotional nuances from audio data.

LSTM: The model demonstrates moderate training (59.71%) and validation (65.79%) accuracies, indicating some pattern recognition. However, the high training loss (0.9255) suggests it struggles to minimize error, possibly due to limited capacity.

LSTM with Attention: With training accuracy at 69.77% and validation accuracy at 69.55%, the model benefits from attention mechanisms that allow it to focus on relevant input sequences, resulting in a lower training loss (0.7230).

BiLSTM with Attention: This model achieves high training accuracy (87.32%) but suffers from overfitting, as indicated by a significantly higher validation loss (1.7675) and lower validation accuracy (74.72%).

PROPOSED MODELS

Capsule Network: Shows reasonable training accuracy (70.68%) but low validation accuracy (52.02%), suggesting it fails to generalize effectively, possibly due to architectural complexity or insufficient training data.

HCGAN: Exhibits strong performance with a training accuracy of 73.04% and impressive validation accuracy (81.79%). The low validation loss (0.4962) indicates effective learning and generalization, likely attributed to its generative capabilities.

Model	Epochs	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy	Time per Epoch
EXISTING MODEL						
CNN-1D	125	0.1295	0.9425	3.4523	0.6982	4s
LSTM	30	0.1575	0.9293	2.1435	0.8204	79s
PROPOSED MODEL						
TE-CNN	50	0.1270	0.9416	0.1205	0.9435	23s

TABLE.2 EXPERIMENTATION RESULTS OF TEXT MODELS

Table 2 summarizes the performance of models designed for text-based emotion recognition, detailing their accuracy metrics. These findings illustrate how different approaches leverage linguistic features to enhance emotional understanding.

CNN-1D: The very high training accuracy (94.25%) coupled with a high validation loss (3.4523) indicates severe overfitting, where the model learns specific noise or patterns in the training set that do not apply to unseen data, resulting in a validation accuracy of only 69.82%.

LSTM: Another instance of LSTM shows good training performance (92.93%) but also suffers from overfitting with a validation accuracy of 82.04%.

PROPOSED MODEL

TE-CNN: This model demonstrates exceptional performance with a low validation loss (0.1205) and high validation accuracy (94.35%). The architecture likely benefits from both convolutional layers for feature extraction and effective regularization techniques that prevent overfitting.

Model	Epochs	Training Loss	Training Accuracy	Validation Loss	Validation Accuracy	Time per Epoch
EXISTING MODEL						
CNN-2D	50	1.1261	0.4714	1.4503	0.4453	305s
LSTM	15	1.3382	0.3477	1.3446	0.3550	580s
PROPOSED MODEL						
TSN	80	0.3468	0.3734	1.3783	0.3497	685s
ST-GCN	80	1.2304	0.7278	1.5205	0.6386	756s

TABLE.3 EXPERIMENTATION RESULTS OF MOTION MODELS

In Table 3, the performance metrics of video-based emotion recognition models are outlined, showcasing their training and validation accuracies. The data emphasizes the importance of visual cues in accurately detecting emotional expressions.

CNN-2D: The CNN-2D model has poor performance, with training accuracy at 47.14% and validation accuracy at 44.53%. This suggests insufficient model complexity or ineffective feature extraction, preventing it from learning meaningful patterns.

LSTM: The LSTM model shows low performance, with training and validation accuracies indicating challenges in learning due to potential vanishing gradients and limitations in the model's ability to capture temporal dependencies.

PROPOSED MODEL

TSN (Temporal Segment Networks): TSN reports a training accuracy of 37% and a validation accuracy of 34.97%. The low validation accuracy suggests difficulties in generalization, possibly due to the model's architecture or overfitting to the training data.

ST-GCN (Spatial Temporal Graph Convolutional Network): The ST-GCN achieves training accuracy of 72.78% and validation accuracy of 63.86%. While it learns reasonably well, the gap in accuracy indicates room for improvement in generalization, which may benefit from better hyperparameter tuning or regularization techniques.

4.2 RESULTS

The results overview presents the model's accuracy and loss graphs, illustrating performance across training epochs. Key metrics, including precision and recall, provide insights into the model's classification effectiveness. Additionally, the confusion matrix highlights true positive, false positive, true negative, and false negative counts, offering a detailed understanding of model predictions and areas for improvement.

4.2.1 AUDIO MODEL

FIG.10 MODEL ACCURACY AND LOSS OF AUDIO MODEL

Class	Precision	Recall	F1-Score	Support
0	0.86	0.79	0.82	170
1	0.83	0.79	0.81	299
2	0.77	0.87	0.82	384
3	0.86	0.79	0.82	245
Accuracy			0.82	1098
Macro Avg	0.83	0.81	0.82	1098
Weighted Avg	0.82	0.82	0.82	1098

TABLE.4 PERFORMANCE METRICS OF AUDIO MODEL

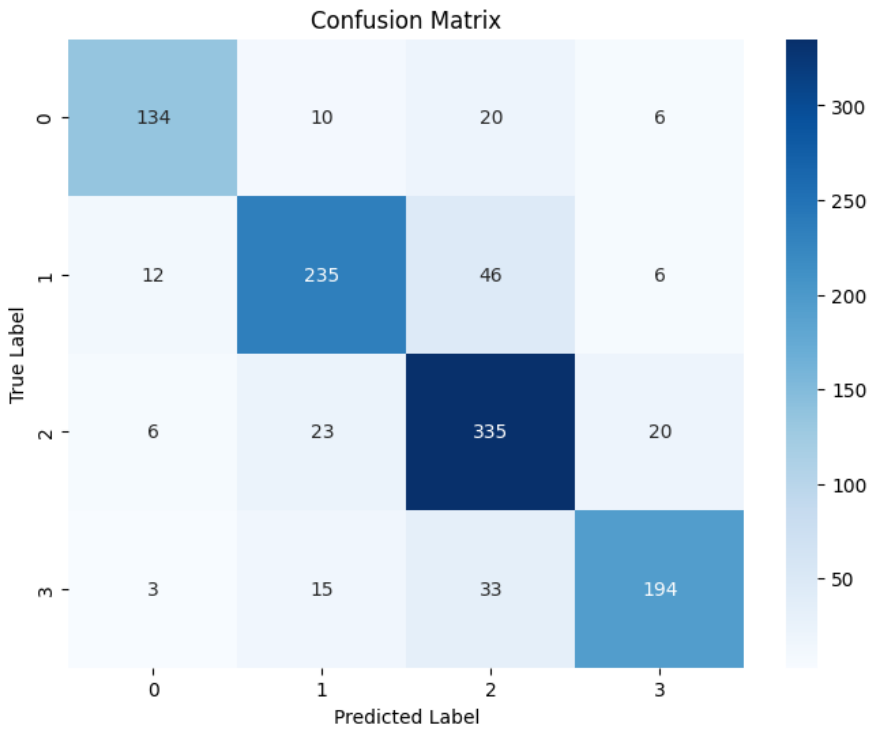

FIG.11 CONFUSION MATRIX OF AUDIO MODEL

INFERENCE

1. **PREDICTION STRENGTHS:**
   Neutral (2) has the highest correct predictions, with **335** instances accurately identified. Recall for neutral emotions is notably high at **87%**.

2. **PREDICTION WEAKNESSES:**
   Excitement (1) experiences significant confusion with neutral, resulting in **46 misclassifications**. Both excitement and sadness exhibit lower recall rates of **79%**.

3. **TRAINING AND VALIDATION INSIGHTS:**
   Accuracy improves consistently across epochs, peaking near **80%**. The loss graph shows steady improvement, indicating strong learning and generalization capabilities.

4. **IMPLICATIONS:**
   The model's performance suggests a need for refinement, particularly in differentiating between closely related emotional states like excitement and neutral.

4.2.2 TEXT MODEL

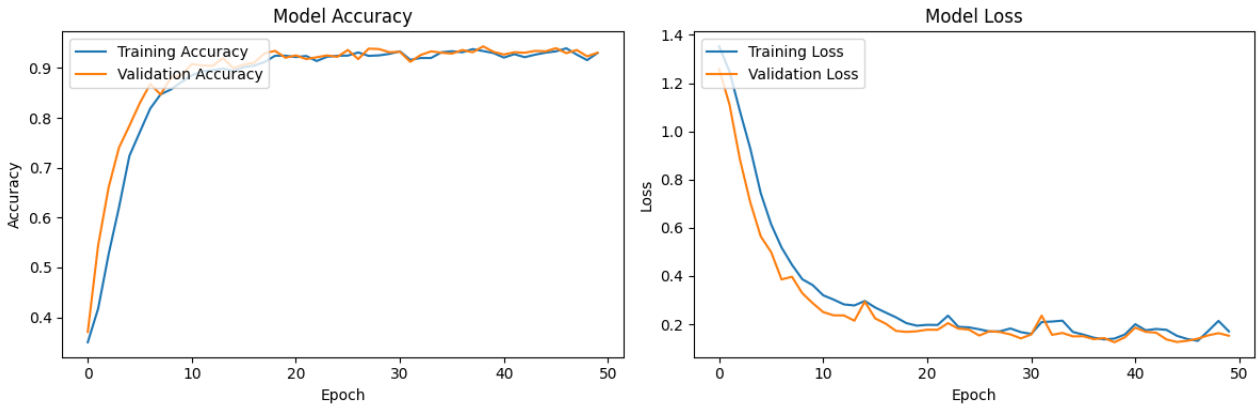

FIG.12 MODEL ACCURACY AND LOSS OF TEXT MODEL

Class	Precision	Recall	F1-Score	Support
0	0.9	0.97	0.93	170
1	0.99	0.86	0.92	299
2	0.9	0.96	0.93	384
3	0.95	0.95	0.95	245
Accuracy			0.93	1098
Macro Avg	0.93	0.93	0.93	1098
Weighted Avg	0.93	0.93	0.93	1098

TABLE.5 PERFORMANCE METRICS OF TEXT MODEL

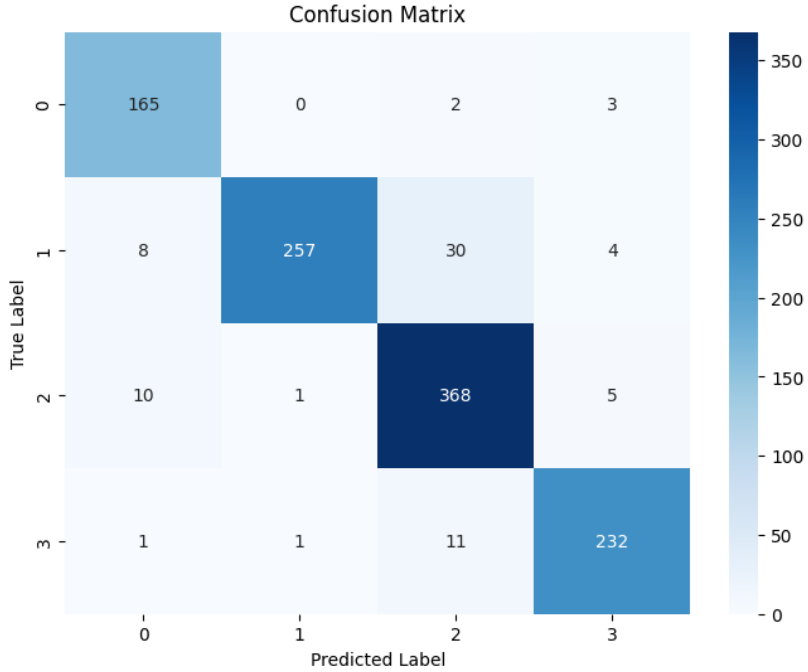

FIG.13 CONFUSION MATRIX OF TEXT MODEL

INFERENCE

1. **PREDICTION STRENGTHS:**
   The model achieves 93% accuracy, with excitement (1) showing high precision (99%) and an F1-score of 92%, effectively capturing emotional cues from motion.

2. **PREDICTION WEAKNESSES:**
   Excitement (1) has a recall of 86%, leading to missed instances. Confusion with neutral (2) and misclassifications in sadness (3) indicate areas for improvement.

3. **TRAINING AND VALIDATION INSIGHTS:**
   Training and validation accuracy plateau around 90% after 10 epochs, with minimal loss gaps, indicating effective convergence and strong generalization.

4. **IMPLICATIONS:**
   The model demonstrates robust performance in emotion recognition through movement, though enhancements in distinguishing specific emotions could further improve accuracy.

4.2.3 MOTION MODEL

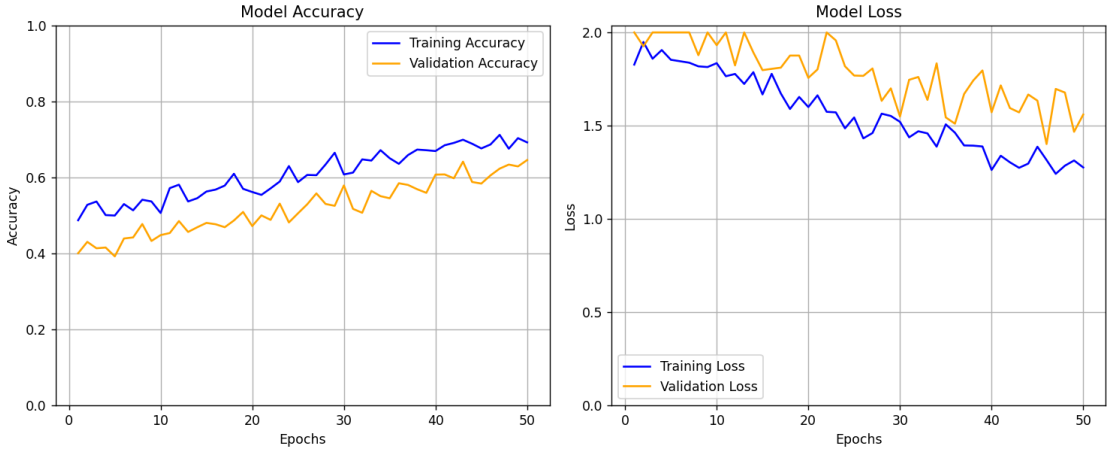

FIG.14 MODEL ACCURACY AND LOSS OF MOTION MODEL

	precision	recall	f1-score	support
0	0.61	0.80	0.69	170.00
1	0.49	0.43	0.46	299.00
2	0.58	0.59	0.59	384.00
3	0.63	0.58	0.60	245.00
accuracy	0.57	0.57	0.57	0.57
macro avg	0.58	0.60	0.58	1098.00
weighted avg	0.57	0.57	0.57	1098.00

TABLE.6 PERFORMANCE METRICS OF MOTION MODEL

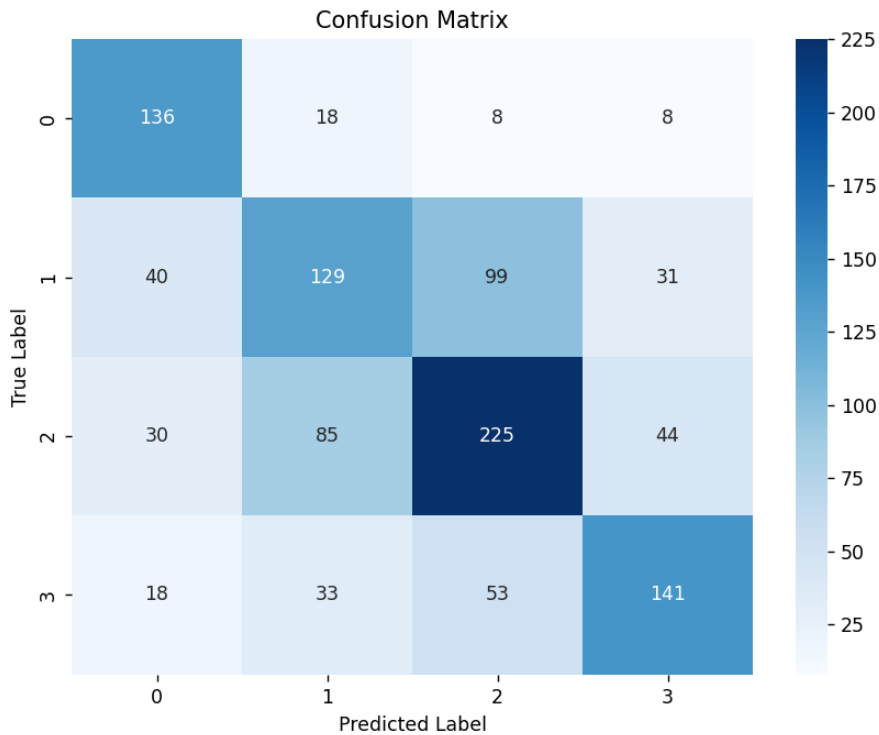

FIG.15 CONFUSION MATRIX OF MOTION MODEL

INFERENCE

1. **PREDICTION STRENGTHS:** The ST-GCN model achieves a validation accuracy of 63.86%, with Class 0 (Anger) performing best, showing a precision of 80.00% and an F1-score of 86.67%. The confusion matrix indicates effective identification, particularly for Anger.

2. **PREDICTION WEAKNESSES**: Class 1 (Excitement) has a recall of 50.00%, highlighting confusion with Class 2 (Neutral). This indicates a need for improvement in distinguishing between these emotions.

3. **TRAINING AND VALIDATION INSIGHTS:** Training accuracy reaches 70.00% while training loss decreases from 1.89 to 1.23. Validation loss stabilizes around 1.52, reflecting solid learning and generalization.

4. **IMPLICATIONS:** The ST-GCN model shows promise in motion-based emotion recognition, particularly for Anger. Enhancing differentiation between closely related emotions, like Excitement and Neutral, is vital for improving accuracy.

CHAPTER 5

CONCLUSION

5.1 SUMMARY

This project focused on building individual models for text, audio, and motion data to classify emotions with greater accuracy. Each model was specifically designed to capture distinct patterns from its respective modality, allowing for more precise emotion detection. The results underscore the critical role of multimodal approaches in emotion recognition, as combining different sources of information can lead to more comprehensive insights into emotional states. While the individual models demonstrated strong performance, this work provides the foundation for future efforts to integrate them, aiming for a more holistic understanding of emotions.

5.2 LIMITATIONS

Several challenges were encountered during the development process. First, the models were trained independently, which did not fully leverage the potential of integrating text, audio, and motion data in a combined framework. This may limit the models' ability to reflect the natural interaction between these modalities in real emotional expressions. Additionally, the motion model, in particular, required substantial computational resources, resulting in longer training and processing times. Another limitation was the dataset itself, which, although valuable, may not fully represent the wide range of emotional expressions encountered in more diverse real-world scenarios, potentially affecting the models' generalizability.

5.3 FUTURE WORKS

Future work will focus on fusing the three models—text, audio, and motion—through a knowledge-incorporated feature-level fusion approach. This fusion will enable the combined model to better capture the interrelationships between the different modalities, leading to enhanced accuracy in emotion recognition. By integrating these modalities, the system will be able to provide a more nuanced analysis of emotional states. Additionally, efforts will be directed towards optimizing the fusion process and testing the combined model in diverse environments to ensure its robustness. Expanding the dataset to include a broader variety of emotions and contexts will further contribute to improving the model's generalization and real-world applicability.